1. Stack Derived From Vector 04A

讲义中实现的Stack，是在底层引入一个Vector，并将其front/rear两端分别指定为bottom/top。

a) 试颠倒front/rear的角色，仿照示例代码实现另一个Stack；

b) 如此改动之后，Stack各接口的性能有何变化？为什么？

2. Stack Derived From List 04A

其实在示例代码中，已经基于List实现了另一个Stack。

a) 试在示例代码包中找到对应的项目，确认List之front/rear两端各自在其中的角色分工；

b) 如果颠倒二者的角色，Stack各接口的性能有何变化？

c) 这些变化是实质的吗？

3. Theseus' Tools 04B1

如果将程序执行过程中的调用跟踪树比作Theseus探索的迷宫，那么调用栈就对应于他所使用的那个线团。然而，其实他需要使用工具还不止于此。比如，为了避免“故地重游”甚至“原地转圈”，他或许还要使用一支粉笔来对已经访问过的位置做标记。C/C++等编程语言中的什么机制，对应于这支“粉笔”？

4. Recursion Depth & Storage Cost 04B1

对于采用分治策略的算法，如果通过优化使得调用栈的每一帧都能节省一半空间，那么可求解问题的规模将会扩大到何等程度？如果能节省到**1/10**呢？估计时，可暂且忽略其他方面的空间消耗。

5. Recursion & Iteration 04B2

a) 是否从理论上讲，程序中的任何递归都可消除掉？

b) 如果不是，原因何在？或者，试举一例；

c) 如果总是可以，消除递归的通用方法是什么？

6. Binary/Linear Recursion 04B2

基于分治策略的算法通常都采用二分递归的模式，每个递归实例都有一前一后的两次递归。如果后一次递归符合尾递归的特征——在对应的递归实例中是最后的一步操作——那么是否也可以套用讲义中的方法予以消除，从而首先将原算法转化为线性递归的模式？

7. Almost-Tail Recursion 04B2

虽然严格地讲，递归版阶乘算法fac()还不是尾递归，却已非常接近：在递归调用之后虽还有一次乘法，但并非实质的计算。这类准尾递归算法也可以做**TCO**，重复使用调用栈中的帧，而不必压入新帧。

a) 试重写该算法，要求彻底甩掉栈，将空间复杂度优化至$\mathcal{O}(1)$；

b) 你针对fac()的去递归的方法，如何推广到一般的准尾递归算法？

8. Head Recursion 04B3

a) 递归版进制转换算法convert()在递归调用之前，已经做过了一次除法。既如此，还能说它是头递归吗？

b) 为什么说，纯粹的头递归（在递归基之后便直接递归，不做任何操作）必然会导致调用栈的溢出？

c) 为什么说对于头递归，调用栈记录的信息中，只有调用参数和输出才是必要的？

9. Destruct A Singly Linked List 03B3

考查所谓的单向链表：节点之间只有succ引用，而没有pred引用。

a) 试实现一个头递归算法，在线性时间内完成单向链表的析构；

b) 你的算法，可能是纯粹的头递归吗？

c) 将你的算法改写为迭代式的，时间复杂度不变；

d) 相对于递归版本，迭代版本在空间成本方面有多大的改进？

10. Customized Parenthesis 04C

试拓展讲义中的括号匹配检测算法，使之能够：

a) 支持（如Html、Jason等格式中）可自定义的任何“括号”；

b) 在判断出失配的同时，还指出错误出现的位置，并给出修正的建议。

11. Priority Table 04D

鉴于加法满足结合律，“**2+3+4**”应该有两种等效的求值方式。

a) 按照evaluate()算法使用的运算符优先级表，将会以哪种方式求值？

b) 如果要改以另一种方式求值，这张表应该如何调整？

c) 算法是否同时还需调整？如需调整，具体如何调整？

d) 如此调整之后，算法的时间、空间复杂度有何变化？

12. Priority Table 04D

乘方运算并不满足结合律，因此“**2^3^4**”的两种求值方式有本质区别。

a) 按照evaluate()算法使用的运算符优先级表，将会以哪种方式求值？

b) 这种方式是否符合数学课上默认的惯例？

c) 如果要改以另一种方式求值，这张表应该如何调整？

d) 算法是否同时还需调整？如需调整，具体如何调整？

e) 如此调整之后，算法的时间、空间复杂度有何变化？

13. Priority Table 04D

a) 在evaluate()算法使用的运算符优先级表中，右括号')'对应的那一行为什么全是空格' '？

b) 该表中还有若干项也是空格' '，它们分别对应于什么情况？

14. Double Factorial 04D

所谓双阶乘，记作$n!!$，等于$[1, n]$范围内与n同奇偶的所有整数的乘积，比如：

```
6!! = 6 * 4 * 2 = 48
5!! = 5 * 3 * 1 = 15
```

为了让evaluate()算法支持双阶乘，我们大致可以采用两类方式：

- 直接方式：将相邻的一对感叹号"!!"理解为双运算符

- 间接方式：新引入一个字符（比如'%'），并约定以之作为双阶乘运算符

a) 试以间接方式，完成对evaluate()算法的功能扩充；

b) 试以直接方式再试试：相对于间接方式，直接方式会多出哪些困难？

15. calcu() 04D

当optr栈顶运算符优先于当前运算符时，evaluate()算法会调用calcu(opnd, optr.pop())执行计算。

a) 试阅读该函数对应的代码，了解该函数是如何区分一目运算与二目运算的；

c) 对于二目运算符，为什么从opnd栈中先取出的是运算数**b**，后取出是**a**？

d) 对于除法运算，除零的情况是如何处理的？你有什么更好的办法？

16. Stack Size In evaluate() 04D

a) 试验证：在evaluate()算法中的任何时刻，optr栈中相邻运算符的相对优先级，都是完全一致的；

b) 试举例说明：上述一致性，并不满足传递律；

c) 既然运算符种类有限，optr栈及opnd栈规模的上限是否为$\mathcal{O}(1)$？

c) 试对于任意的$n \gg 2$构造一个表达式，使得evaluate()算法中optr栈的最大规模可以达到n；

d) 为此，表达式中至少需要有多少对括号？

17. RPN Convert **04E**

a) 试证明：按照讲义中介绍的手工方法，的确可将任意的常规表达式转换为RPN；

b) 试证明：evaluate()算法转换RPN的算法，也是正确的；

c) 尽管最终的效果相同，这两种算法的转换过程有何区别？

18. RPN **04E**

我们看到，**RPN**的求值算法要比常规表达式更加简明、高效。然而我们也注意到，无论是手工地还是自动地完成从后者到前者的转换，本身也需要线性的时间。既如此，RPN的价值何在？

19. RPN **04E**

试分别将以下常规表达式，转换为对应的RPN：

- (0+1+2-3+4+5+6)*(7+8)*9
- (0*1+2)*3!/4!*5*6*(7+8)*9
- 0!+(1+2)!!*3-4-5!-6-7-8+9
- (0+1+2)/3!/4!*5!*6*(7+8)*9
- (0!^1/2*3!!/(4+5)*6-7-8)*9
- ((0*(1-2)+3+4)*5*6+(7+8))*9
- (0!-1!+2)*3!/4!*5*6*(7+8)*9
- ((0-1)^2*(3!!/4-5+6*7+8))*9
- ((0-1)^2*(3!!/4-5-6+7*8))*9
- (0!+1+2)!+(3!-4)^(5+6)-7*8+9
- (0+(1+2))^3/(4+5)*6!-(7+8)*9
- ((0-1)^2!*(3!!/4-5+6*7+8))*9
- ((0-1)^2!*(3!!/4-5-6+7*8))*9
- (0!+1)*(2+3)*(4+5)/6*(7+8)*9
- (0!^1/2*3!!/(4+5)*6-(7+8))*9
- ((0-1)^2*(3!!/4-5*(6-7-8)))*9
- ((0-1)^2!*(3!!/4-5*(6-7-8)))*9
- ((0!+1!)^2+(3!-4)+5*6)*(7*8)+9
- ((0!+1)*2)!+(3!-4)^(5+6)-7*8+9
- ((0!^1+2)!!/(3!-4-5+6)-(7+8))*9
- ((0!^1+2)!!/(3!-(4+(5-6)))-(7+8))*9

20. Polish Natation **04E**

a) 如果采用**PN**（运算符均前置而非后置）来做表达式求值，是否也能如**RPN**那样同样地简明、高效？

b) 如果可以，试给出对应的求值算法；否则，试解释原因。

21. Stack Permutation: Forbidden Patterns **04F**

试证明：

a) 含有“$\ldots k \ldots i \ldots j \ldots$”禁形的任何序列，必然不是栈混洗序列；

b) 含有“$\ldots j+1 \ldots i \ldots j \ldots$”禁形的任何序列，必然不是栈混洗序列。

22. Stack Permutation: Yet Another Counting **04F**

关于栈混洗的计数，课上所给出的递推式，是根据“A栈顶元素何时进入B栈”来做分类。

a) 试变换视角，依据“A栈中哪个元素最后进入B栈”来做分类，从而导出另一个递推式。

b) 改成看“A栈中哪个元素率先进入B栈”呢？

23. CSTDSA **04F**

考查栈<**CSTDSA**]的混洗序列。其中两个字符**S**，不妨以$\mathbf{S_a}$和$\mathbf{S_b}$来区别。

a) 试验证：形如[*****$\mathbf{S_a}$>的混洗序列必然形如[**C******$\mathbf{S_a}$>；

b) 形如[**C******$\mathbf{S_a}$>的混洗序列共有多少个？

c) 试验证：形如[*****$\mathbf{S}_b$>的混洗序列必然形如[****$\mathbf{AS}_b$>；

d) 形如[****$\mathbf{AS}_b$>的混洗序列共有多少个？

e) 形如[**C***AS**$_a$>和[**C***AS**$_b$>的混洗序列，各有多少个？

f) 形如[**C***AS**$_a$>和[**C***AS**$_b$>的混洗序列之间，不同、相同的各有多少个？

g) 形如[*****S**>混洗序列，共有多少个？

24. Stack Permutation: Identification 04F

本节的spCheck(Vector<Rank>& B)算法，可以通过模拟混洗过程来甄别栈混洗序列。

a) 试证明：该算法确实正确——它的返回值小于n，当且仅当B不是栈混洗序列。

该算法的主体是二重循环：外层for，内层while。

b) 试证明：尽管for循环共需迭代n步，while循环在最坏情况下也需迭代$\Omega(n)$步，但总体只需$\mathcal{O}(n)$时间。

25. Stack Permutation vs. Parenthesis 04F

a) 按照讲义及课上讲解的思路严格证明：n对括号构成的合法表达式，与n个元素的栈混洗一一对应；

b) 你是否还能从其他角度，来理解并证明这一对应关系？

26. Parenthesis Expression 04F

不难理解，由n对括号构成的所有合法表达式，可按字典序从1到catalan(n)编号。

试阅读D. E. Knuth的The Art Of Computer Programming第4卷第4册，了解以下问题及算法：

a) Algorithm P (I. Semba, 1981)：按字典序枚举由n对匹配括号组成的所有表达式；

b) Algorithm U (F. Ruskey, 1978)：按字典序取出其中的第k个；

c) Algorithm W (D. B. Arnold & M. R. Sleep, 1980)：等概率地从中随机选出一个。

27. Queue Derived From Vector 04G

a) 讲义中选择了基于列表来实现队列，是否也可改为基于向量来实现？

b) 如果可以，具体如何实现？

c) 如果不可以，原因何在？

28. More Queue Applications 04H

除了讲义中所举的实例，你还知道在哪些计算过程中使用了这种结构？

29. Stacks As Queue 04I

讲义中以“双栈当队”为例，介绍了均摊分析的三种典型方法。

然而从字面上看，它们所得复杂度的常系数还是有所区别。这种区别仅仅是常系数的放缩，还是本质的？

30. Maximum Rectangle In A Histogram 04J

考查直方图中最大矩形问题。

a) Brute-force算法在最坏情况下需要运行多少时间？试举出这样的实例；

不难理解，最大矩形可能同时存在多个。为此，讲义中约定着眼于其中的最靠左（leftmost）者。

b) 试明确地给出这个“最靠左”的定义；

c) 蛮力算法、两趟扫描算法与单趟扫描算法返回的最大矩形，是否符合上述约定？

d) 在对应的**Excel**演示程序中，找出的最大矩形是否符合上述约定？

31. 2-Pass vs. 1-Pass 04J

直方图中最大矩形问题的单趟扫描算法与两趟扫描算法，前者的哪些特性，是后者并不具备的？

32. Amortized Time Cost Of 1-Pass 04J

教材中采用记账法，界定了两趟扫描算法的总体运行时间为$\mathcal{O}(n)$。

a) 试用模仿上述记账法，证明单趟扫描算法总体也仅需$\mathcal{O}(n)$时间；

b) 试分别采用聚合法、势能法，证明上述结论。

33. Expected Storage Cost Of 1-Pass 04J

考查直方图中最大矩形问题的单趟扫描算法。

a) 在最坏情况下，它的空间复杂度是多少？

b) 在最好情况呢？

c) 试验证：当算法执行到第r步迭代时，若将它即时所占用的空间量记作S_r，则整个算法的空间复杂度应为：

$$\mathrm{S}(n) = \max\{\mathrm{S}_r \,|1 \le r \le n\}$$

d) 试验证教材中所指出的，S_r的数学期望为$\mathbb{E}(\mathrm{S}_r) = \mathcal{O}(\log r)$；

e) 那么，整个算法期望的空间复杂度$\mathbb{E}(\mathrm{S}(n))$是多少？

34. Expected Time Complexity Of MaxQueue 04K

a) 试验证教材中运用聚合法，对MaxQueue结构均摊时间复杂度的分析过程及结论；

b) 试改用记账法，完成这一分析；

c) 试改用势能法，完成这一分析。

35. Expected Storage Complexity Of MaxQueue 04K

我们知道，MaxQueue结构的空间复杂度，决定于增设的压缩版影子队列Y。

这个空间复杂度的数学期望是多少？